

FACULTAD DE INGENIERÍA
Universidad de Buenos Aires

75.29 / 95.06

Teoría de Algoritmos 1 - 1c 2025

Trabajo Práctico N° 2 - 2da Entrega

ACORN

Fecha de Entrega:

9 de junio de 2025

Integrantes:

Alumno	Padrón
Rogger Aldair Paredes Tavara	97976
Santiago Nahuel Ruiz Sugliani	106768
Maria Mercedes Slepowron Majowiecki	109454
Jose Ignacio Adelardi	111701

Parte 1: La Subasta de Pizza.....	4
1. Explique brevemente cada una de las soluciones. Defina la función costo y límite cuando corresponda.....	4
2. Dar el pseudocódigo y estructuras de datos a utilizar en ambas propuestas.....	7
3. Realice el análisis de complejidad temporal y espacial de ambas soluciones. Compararlas.....	10
4. Brinde un ejemplo simple paso a paso del funcionamiento de las soluciones.....	12
5. Programe ambas propuestas.....	16
6. Determine si los programas tienen la misma complejidad que su propuesta teórica.....	16
Parte 2: El traslado de Información Secreta.....	18
1. Generar una propuesta mediante redes de flujo que solucione el problema. Explicar la idea de esta.....	18
2. Presentar Pseudocódigo.....	18
3. Realizar un análisis de optimalidad.....	20
4. Realizar análisis de complejidad temporal y espacial. Considere las estructuras de datos que utiliza para llegar a estos.....	20
Análisis de Complejidad.....	20
Complejidad temporal.....	20
Complejidad espacial.....	21
5. Programar la solución. Incluya la información necesaria para su ejecución. Compare la complejidad de su algoritmo con la del programa.....	21
Análisis de Complejidad del Algoritmo.....	21
Complejidad Temporal.....	21
Complejidad Espacial.....	22
6. ¿Es posible expresar su solución como una reducción polinomial? En caso afirmativo explique cómo y en caso negativo justifique su respuesta.....	22
Parte 3: Las 2 Jornadas de Capacitación.....	23
1. Demostrar que, dada una posible solución que nos brindan, se puede fácilmente determinar si se puede cumplir o no con la tarea solicitada.....	23
2. Demostrar que si desconocemos si es posible lograr una solución, es difícil poder afirmar que existe. Utilizar para eso el problema “Set splitting problem” (suponiendo que sabemos que este es NP-C).....	24
3. Demostrar que el problema “Set splitting problem” pertenece a NP-C. (Para la demostración se le solicita investigar. La misma se puede realizar partiendo desde “3-SAT” y pasando por “NAE-3-SAT”)......	24
4. Una persona afirma tener un método eficiente para responder si es posible o no cualquiera sea la instancia. Utilizando el concepto de transitividad y la definición de NP-C explique qué ocurriría si se demuestra que la afirmación es correcta.....	27
5. Un tercer problema al que llamaremos X se puede reducir polinomialmente al problema de “Las 2 jornadas de capacitación”, qué podemos decir acerca de su complejidad?.....	27
Correcciones.....	28
Parte 1.....	28
Función costo - Branch and Bound.....	28
Pseudocódigo - backtracking.....	28
Pseudocódigo - Branch & Bound.....	30
Ejemplo paso a paso - Branch & Bound.....	33

Corrección en el programa desarrollado - Branch & Bound.....	35
Análisis de complejidad real.....	35
Parte 3.....	36
Pregunta 2 - Demostrar que si desconocemos si es posible lograr una solución, es difícil poder afirmar que existe. Utilizar para eso el problema “Set splitting problem” (suponiendo que sabemos que este es NP-C).....	36
Pregunta 3 - parte 2: Reducción de 3-SAT a NAE-3-SAT.....	37
Pregunta 5: Un tercer problema al que llamaremos X se puede reducir polinomialmente al problema de “Las 2 jornadas de capacitación”, qué podemos decir acerca de su complejidad?..	37

Parte 1: La Subasta de Pizza

1. Explique brevemente cada una de las soluciones. Defina la función costo y límite cuando corresponda.

Solución por Backtracking

Se propone una solución por backtracking planteando un árbol combinatorio para representar al árbol de estados del problema. La raíz del árbol representa la mesa vacía. Se va a formar un árbol de m niveles (a lo sumo) donde cada uno representa la porción de pizza a asignar. En cada nivel del árbol, los nodos representan al invitado que tomará esa porción, asociando al nodo la oferta que ese invitado n realiza por esa porción de pizza i . Los descendientes de cada nodo corresponden a asignarle la porción $i+1$ al siguiente invitado, posterior a haber seleccionado a dicho nodo. Al agregar un invitado, ocupa ese lugar en la mesa y brinda la oferta para la porción en esa posición. Cada nodo estado corresponde a tener en la mesa a cada uno de los invitados desde la raíz hasta ese mismo.

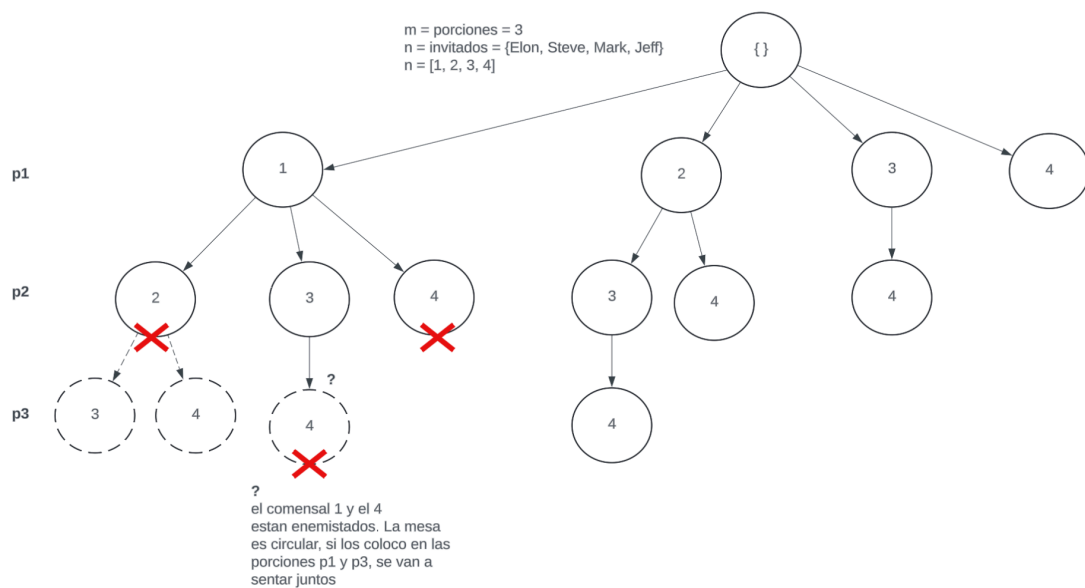
Las restricciones explícitas corresponden a la asignación posible de invitados a cada porción de pizza; cada invitado debe elegir una única porción si se va a sentar en la mesa. La solución no necesariamente debe ser una tupla de m invitados, es posible que queden asientos vacíos en la mesa si eso significa que se maximizará la ganancia, que es el objetivo del restaurante.

Las restricciones implícitas corresponden a evitar que dos o más invitados enemistados se sienten de manera contigua. Eso incluye la consideración que el invitado en la posición 1 de la mesa no puede ser enemigo del invitado en la posición m , dado que es una mesa circular. El criterio de optimización es maximizar la ganancia del restaurante según la oferta de cada invitado sobre la pizza que se encuentra en el asiento que va a tomar.

Teniendo en cuenta la restricción de invitados enemistados, dado que los invitados pueden decidir no sentarse al lado de uno o varios invitados en particular, utilizaremos una función límite que nos permita podar las ramas del árbol en donde haya asignaciones inválidas. Es decir, podaremos los nodos descendientes si ese invitado está enemistado con el invitado representado en el nodo padre (el que se sienta contiguo a él).

Ejemplificando de manera visual, supongamos que tenemos $m=3$ porciones de pizza para repartir entre $n=4$ comensales: Elon, Steve, Mark y Jeff. Sabemos que Elon no se quiere sentar junto a Jeff, y que Steve no se quiere sentar junto a Elon.

Ordenamos arbitrariamente a los invitados por su orden de llegada (el orden en el archivo). Le asignamos a cada uno un identificador numérico, de tal forma que al colocar un elemento i en el árbol, los nodos sucesores a este corresponden a los elementos $[i+1, n]$. Esto se realiza para evitar duplicaciones, ya que las rotaciones de mesa no se evaluarán en el backtracking. (No vamos a evaluar si podemos sentar 1ro a Elon y después a Jeff, o primero a Jeff y después a Elon también. Obtenemos solo una de esas configuraciones).



En este planteo, estamos armando el árbol de estados para obtener únicamente las combinaciones posibles, sin tener en cuenta las rotaciones. Por ejemplo: vamos a establecer un caso donde los invitados ofertan la siguiente cantidad de dinero por cada porción:

invitado/número de porción	1	2	3
Elon (1)	2	5	10
Steve (2)	3	4	5
Mark (3)	10	1	0
Jeff (4)	7	3	2

Observando el árbol de estados, tenemos como la primera configuración factible a $[1, 3, -]$, que nos daría una ganancia total de $2 + 1 = 3$. Consideremos entonces las rotaciones posibles para esos comensales; se presentan:

- $[-, 1, 3]$ donde obtenemos la ganancia $0 + 5 + 0 = 5$ una ganancia mejor a la obtenida originalmente por el árbol de decisiones.
- $[3, -, 1]$ donde obtenemos la ganancia $10 + 0 + 10 = 20$ una ganancia aún mejor a las anteriores y por lo tanto la mejor ganancia obtenida hasta el momento

Observemos la segunda configuración factible $[2, 3, 4]$:

- $[2, 3, 4]$ con ganancia $3 + 1 + 2 = 6$.
- $[4, 2, 3]$ con ganancia $7 + 4 + 0 = 11$
- $[3, 4, 2]$ con ganancia $10 + 3 + 2 = 15$

Observemos la tercera configuración factible $[2, 4, -]$

- $[2, 4, -]$ con ganancia $3 + 3 = 6$

Observemos la cuarta configuración factible $[3, 4, -]$

- $[3, 4, -]$ con ganancia $10 + 3 = 13$

Finalmente la quinta configuración factible [4, -, -] da una ganancia de 7.

El cálculo de las rotaciones se realiza posteriormente al backtracking. Es por ello que tampoco incluimos la ganancia que trae cada comensal para esa porción de pizza.

Solución por Branch and Bound

Se propone una solución por Branch and Bound, planteando un árbol combinatorio para representar el espacio de estados del problema. La raíz del árbol representa la mesa completamente vacía, sin porciones asignadas. Se construye un árbol de a lo sumo m niveles, donde cada nivel representa la asignación de una porción de pizza. En cada nivel, los nodos representan a un invitado que se sienta en la posición correspondiente a una porción y realiza su oferta para esa porción.

Cada nodo contiene el estado parcial actual: qué invitados están sentados y en qué lugar, junto con la ganancia acumulada hasta el momento. Los descendientes de un nodo representan la posibilidad de asignar la siguiente porción (posición $i+1$) a un nuevo invitado que aún no haya sido asignado, generando así nuevas configuraciones parciales del problema. Al agregar un invitado a una porción, se lo sienta en ese lugar de la mesa circular, y se asocia la oferta correspondiente a la ganancia acumulada.

La solución completa (hoja del árbol) puede tener menos de “ m ” invitados, ya que algunos lugares pueden quedar vacíos si eso permite mejorar la ganancia total del restaurante. Cada invitado puede obtener como mucho una porción.

Las restricciones explícitas indican que un invitado solo puede sentarse una vez, en una única porción. Las restricciones implícitas provienen de la enemistad entre algunos invitados. Si dos invitados asignados a porciones adyacentes están enemistados, entonces la asignación no es válida. Esta condición se debe verificar también para los extremos de la mesa, ya que es circular: la primera y la última porción son contiguas.

Dado que el espacio de soluciones es exponencial (de orden $O(n! / (n-m)!)$), se implementa una función límite (bound) que estima de forma optimista la mejor ganancia posible a partir de un estado parcial. La estrategia de Branch and Bound permite priorizar, expandir y eventualmente poder descartar (poda) aquellas ramas del árbol que:

- Violan las restricciones de enemistad: contiene invitados enemistados en porciones adyacentes
- No puedan superar la mejor ganancia encontrada hasta el momento:
 $bound(A) \leq \text{mejor ganancia actual}$

Se propone utilizar una cola de prioridad (Max-Heap) para expandir siempre primero el nodo con mayor bound, lo que corresponde a un recorrido tipo Best-First Search.

Función Costo

Queremos sobreestimar la ganancia posible, maximizando la misma. Para cada porción, sumamos la máxima oferta de cualquier invitado.

Se propone la función de costo:

$$\text{bound}(A) = g(A) + \sum_{p \in \text{porciones restantes}} \text{max oferta}_{i,p}$$

con $g(A)$ ganancia de la asignación actual, e $i \in \text{invitados no sentados}$

Por ejemplo: tenemos $m = 2$, invitados $I = [I1, I2, I3]$

invitado/número de porción	0	1
Elon (I1)	10	3
Steve (I2)	4	8
Mark (I3)	6	6

bound = 0

para la porción 0: $\max(10, 4, 6) = 10$

para la porción 1 P1: $\max(3, 6, 6) = 6$

bound = $10 + 6 = 18$

Asignamos este bound al nodo raíz, para arrancar.

2. Dar el pseudocódigo y estructuras de datos a utilizar en ambas propuestas.

Pseudocódigo para la solución de Backtracking

```
// E: Diccionario donde E[i] es el conjunto de invitados con los que i está enemistado.
// P: Diccionario donde P[i][j] es la oferta del invitado i por la porción j.
// n: Número de invitados.
// m: Número de porciones (n > m).
// Retorna la asignación de invitados a porciones que maximiza la ganancia.
```

```
UbicacionesFactibles(invitados, E, P, n, m)
  uFactibles = []
  ObtenerUbicaciones(invitados, E, [], [], 0, n, m, uFactibles)
  Retornar MaximaGananciaConRotaciones(uFactibles, P, m)
```

```
ObtenerUbicaciones(invitados, E, uParcial, i, n, m, uFactibles)
  Si len(uParcial) == m
    Si EsFactibleCircular(uParcial, E)
      uFactibles.append(uParcial.copy())
  Retornar
```

```

Si i >= n
    uFactibles.append(uParcial.copy())
    Retornar

Si PuedoUbicarlo(uParcial, invitados[i], E)
    uParcial.append(invitados[i])
    ObtenerUbicaciones(invitados, E, uParcial, i+1, n, m, uFactibles)
    uParcial.pop()
    ObtenerUbicaciones(invitados, E, uParcial, i+1, n, m, uFactibles)

PuedoUbicarlo(uParcial, invitadoActual, E)
    Si len(uParcial) == 0
        Retornar Verdadero

    invitadoAnterior = uParcial[-1]
    Si invitadoActual está en E[invitadoAnterior]
        Retornar Falso

    Retornar Verdadero

EsFactibleCircular(uParcial, E)
    primerInvitado = uParcial[0]
    ultimoInvitado = uParcial[-1]
    Si ultimoInvitado está en E[primerInvitado]
        Retornar Falso
    Retornar Verdadero

MaximaGananciaConRotaciones(uFactibles, P, m)
    gananciaMaxima = 0
    asignacionOptima = []

    Para cada ubicacion en uFactibles
        Para i desde 0 hasta m-1
            rotacion = Rotar(ubicacion, i)
            ganancia = 0
            Para j desde 0 hasta m-1
                invitado = rotacion[j]
                ganancia += P[invitado][j]
            Si ganancia > gananciaMaxima
                gananciaMaxima = ganancia
                asignacionOptima = rotacion
    Retornar (gananciaMaxima, asignacionOptima)

Rotar(ubicacion, k)
    n = len(ubicacion)
    Retornar ubicacion[k:n] + ubicacion[0:k]

```

Pseudocódigo para Branch and Bound


```
BranchAndBound(m, n, ofertas, enemistades):
    mejor_ganancia = 0
    mejor_asignacion = []

    nodo_raiz = CrearNodo(
        asignaciones = [],
        ganancia_actual = 0,
        invitados_restantes = {0..n-1},
        porciones_restantes = {0..m-1},
        bound = costo([], 0, {0..n-1}, {0..m-1}, ofertas)
    )

    asignacion = MaxHeap()
    insertar(asignacion, nodo_raiz)

    mientras no esta_vacio(asignacion):
        nodo ← extraer_max(asignacion)

        si nodo.bound ≤ mejor_ganancia:
            continuar # podemos

        si size(nodo.porciones_restantes) == 0:
            si nodo.ganancia_actual > mejor_ganancia:
                mejor_ganancia ← nodo.ganancia_actual
                mejor_asignacion ← nodo.asignaciones
            continuar

        porción_actual ← seleccionar_porcion(nodo.porciones_restantes)

        para cada invitado en nodo.invitados_restantes:
            si es_valida_asignacion(nodo.asignaciones, porción_actual, invitado,
enemistades, m):
                nueva_asignación ← copiar(nodo.asignaciones)
                nueva_asignación[porción_actual] ← invitado

                nueva_ganancia ← nodo.ganancia_actual + ofertas[invitado][porción_actual]

                nuevos_invitados ← nodo.invitados_restantes - {invitado}
                nuevas_porciones ← nodo.porciones_restantes - {porción_actual}

                nuevo_bound ← costo(nueva_asignación, nueva_ganancia, nuevos_invitados,
nuevas_porciones, ofertas)

                nuevo_nodo ← CrearNodo(
                    asignaciones = nueva_asignación,
                    ganancia_actual = nueva_ganancia,
                    invitados_restantes = nuevos_invitados,
                    porciones_restantes = nuevas_porciones,
```

```

        bound = nuevo_bound
    )

    insertar(heap, nuevo_nodo)

    retornar mejor_ganancia, mejor_asignacion

costo(asignaciones, ganancia_actual, invitados_restantes, porciones_restantes, ofertas):
    bound ← ganancia_actual

    para cada porción p en porciones_restantes:
        mejor_oferta ← 0
        para cada invitado i en invitados_restantes:
            mejor_oferta ← max(mejor_oferta, ofertas[i][p])
        bound ← bound + mejor_oferta

    retornar bound

es_valida_asignacion(asignaciones, porcion_actual, invitado, enemistades, m):
    vecinos ← {
        (porcion_actual - 1 + m) mod m,
        (porcion_actual + 1) mod m
    }

    para cada vecino en vecinos:
        si vecino ∈ asignaciones:
            invitado_vecino ← asignaciones[vecino]
            si (invitado, invitado_vecino) ∈ enemistades o (invitado_vecino, invitado) ∈
enemistades:
                retornar falso

    retornar verdadero

```

3. Realice el análisis de complejidad temporal y espacial de ambas soluciones. Compararlas.

Complejidad temporal por backtracking

En el caso de la solución por backtracking, necesitamos explorar todas las combinaciones posibles de asignación de invitados a porciones. Tenemos un caso con m porciones y n invitados, donde tenemos que probar todas las combinaciones posibles de asignar a cada invitado una porción.

Al realizar esta evaluación en `ObtenerUbicaciones()` tenemos $\binom{n}{m}$ combinaciones, y para cada combinación de m invitados hay hasta $m!$ permutaciones. En el peor caso, no hay enemistades y no podemos podar ninguna de las permutaciones. Llegamos a un costo de obtener todas las ubicaciones

exploradas de: $O\left(\binom{n}{m} \cdot m!\right)$. Se continúa evaluando `EsFactibleCircular`, que se ejecuta una sola vez por ubicación completa ($\text{len}(uParcial) == m$) y compara los extremos de la lista (primer y último invitados). Esto se puede hacer en un tiempo constante $O(1)$. Para cada ubicación factible U , se exploran m rotaciones (una por cada porción disponible), y para cada rotación se calcula la suma de ofertas (ganancia) correspondiente. Tenemos un costo de cálculo por ubicación de $O(m^2)$ (m rotaciones * m porciones para calcular la ganancia). Este costo se calcula para las U ubicaciones factibles, las cuales se pueden acotar por las $\binom{n}{m} \cdot m!$ combinaciones. Por último, el método `Rotar()` rota una lista de tamaño m , lo cual se puede realizar en tiempo lineal, con costo: $O(m)$.

Obtenemos finalmente, una complejidad espacial: $O\left(\binom{n}{m} \cdot m! \cdot m^2\right)$

Complejidad espacial por backtracking

Desde el punto de vista espacial, el algoritmo almacena una lista con todas las ubicaciones factibles generadas, cada una de tamaño m . En el peor caso, se pueden almacenar hasta $\binom{n}{m} \cdot m!$ configuraciones. Además, se utilizan variables auxiliares temporales de tamaño m durante el proceso de backtracking y evaluación de rotaciones. Por lo tanto, la complejidad espacial está dada por $O\left(\binom{n}{m} \cdot m! \cdot m\right)$. Este espacio corresponde al almacenamiento de todas las combinaciones válidas de invitados y sus respectivas rotaciones.

Complejidad temporal por Branch and Bound

Sin tener en cuenta las restricciones de enemistad, el número total de asignaciones posibles viene dado por las permutaciones de los m invitados elegidos de un conjunto de n invitados totales: $P(n, m) = \frac{n!}{(n-m)!}$. En consecuencia, y sumando también los nodos intermedios, el tamaño del árbol en el peor de los casos es: $O(P(n, m)) = O\left(\frac{n!}{(n-m)!}\right)$

Cada vez que extraemos y expandimos un nodo del heap, realizamos las siguientes operaciones:

- *Cálculo de cota (función costo)*: Recorremos cada una de las porciones restantes (como máximo m) y, para cada una, examinamos todas las ofertas de los invitados que aún no han sido asignados (hasta n). Esto tiene una complejidad: $O(m * n)$
- *Comprobación de asignación válida*: Sólo debe verificar como máximo dos vecinos (las porciones adyacentes) y consultar en una estructura de datos de acceso constante (por ejemplo un set o hash) si existe enemistad. Esto tiene una complejidad: $O(1)$
- *Operar con un heap max*: Insertar o extraer un nodo de un heap de tamaño k cuesta $O(\log k)$. En el peor de los casos, el heap puede crecer hasta contener los $P(n, m)$ nodos, por lo que cada operación sería $O(\log P(n, m)) = O(\log(n!)) = O(n \log n)$

Sumando los costos de bound y las operaciones de heap, cada expansión tiene un costo: $O(mn + n \log n)$. Como el árbol puede llegar a tener $\frac{n!}{(n-m)!}$ nodos, la cota de complejidad temporal resulta: $O\left(\frac{n!}{(n-m)!}\right) * (mn + n \log n)$

Complejidad espacial por Branch and Bound

Por cada nodo contamos con un vector de asignaciones parciales (tamaño m), un conjunto de invitados y porciones restantes (tamaño $\leq n$ y $\leq m$ respectivamente) y un conjunto de variables escalares (bound y ganancia). Por lo tanto, el espacio necesario para cada nodo resulta ser $O(n + m)$. Dado que el árbol puede llegar a tener $\frac{n!}{(n-m)!}$ nodos, la cota de complejidad temporal resulta: $O\left(\frac{n!}{(n-m)!}\right) * (n + m)$

4. Brinde un ejemplo simple paso a paso del funcionamiento de las soluciones

Paso a Paso de solución por Backtracking

Supongamos que tenemos los siguientes datos de entrada:

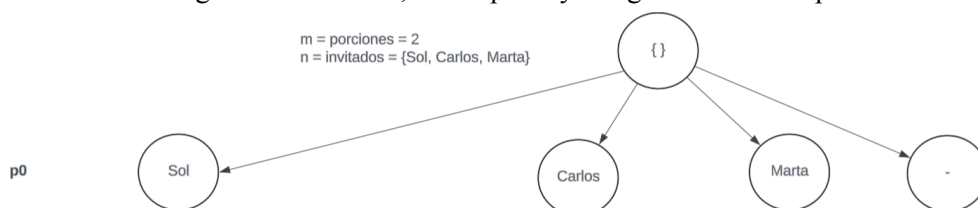
invitado/número de porción	1	2
Sol (1)	10	5
Carlos (2)	5	10
Marta (3)	8	8

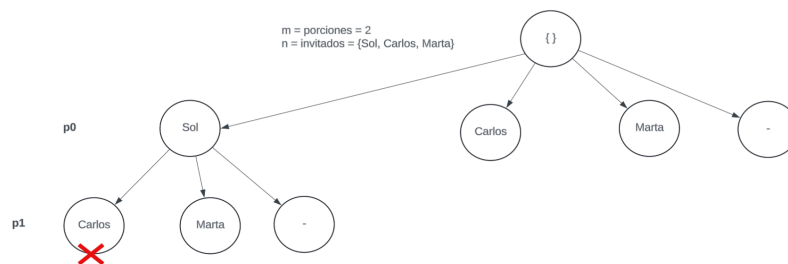
Enemistades: Sol no quiere sentarse con Carlos

Generamos un árbol con $m=2$ niveles (uno por porción). A cada porción se le puede asignar, o un invitado no usado aún, que a su vez no esté enemistado con el invitado del nivel anterior, o dejar una porción vacía.

Nivel 0 - Porción 0

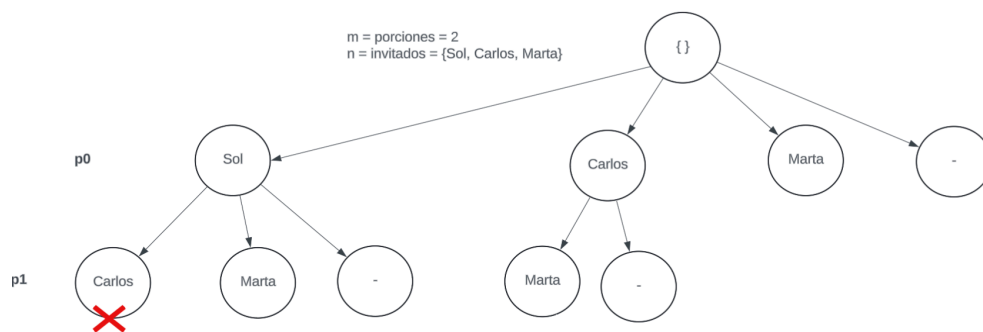
Tenemos las siguientes asignaciones válidas a esa porción. Recorremos en profundidad cada rama para encontrar al siguiente comensal, hasta que haya asignado todas las porciones.





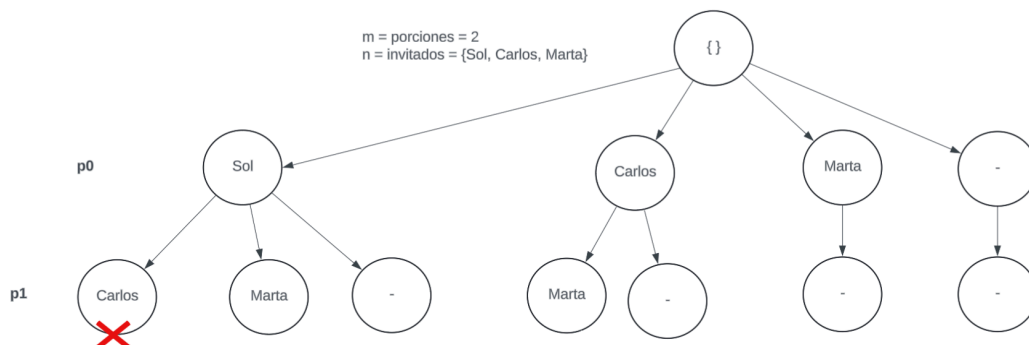
Teniendo a Sol en el asiento de la porción 0 intentamos sentar a Carlos con la porción 1. Están enemistados, así que no es una asignación posible. Volvemos para atrás e intentamos asignar a alguien más con la porción 1. Asignamos a Marta, y como no están enemistadas se genera la asignación posible: [Sol, Marta]; la guardamos. Volvemos para atrás e intentamos asignar la porción 1 a alguien más. Dejar el asiento vacío es una asignación válida, así que generamos la asignación posible: [Sol, -].

Este mismo recorrido lo seguimos para evaluar a Carlos, y sus posibles asignaciones. Como ya sentamos a Sol, sentamos al resto de los comensales que nos quedan por procesar después de Carlos nada más. En este caso, Marta:



Le asignamos a Marta la porción 1, y como no están enemistados se genera la asignación posible: [Carlos, Marta]; la guardamos.

Siendo Marta la última comensal por evaluar, la única asignación posible que le queda para la porción 1 es que no se siente nadie. Se genera la asignación [Marta, -]



Habiendo recorrido todo el árbol, obtuvimos todas las asignaciones posibles:

- [Sol, Marta]
- [Sol, -]
- [Carlos, Marta]
- [Carlos, -]
- [Marta, -]

Para cada una de ellas, rotamos sus elementos y nos quedamos con el de mayor ganancia total:

asignación/rotación	ganancia
[Sol, Marta]	$10 + 8 = 18$
[Marta, Sol]	$8 + 5 = 13$
[Sol, -]	$10 + 0 = 10$
[-, Sol]	$0 + 5 = 5$
[Carlos, Marta]	$5 + 8 = 13$
[Marta, Carlos]	$8 + 10 = 18$
[Carlos, -]	$5 + 0 = 5$
[-, Carlos]	$0 + 10 = 10$
[Marta, -]	$8 + 0 = 8$
[-, Marta]	$0 + 8 = 8$
[-, -]	$0 + 0 = 0$

Paso a Paso de solución por Branch and Bound

Supongamos que tenemos el siguiente problema de ejemplo:

- Porciones $m = 2$ (p_0, p_1)
- Invitados $n = 3$ (I_0, I_1, I_2). Sabemos que I_0 no quiere sentarse al lado de I_1 .

Los invitados realizan las siguientes ofertas:

invitado/número de porción	p_0	p_1
I_0	10	3
I_1	4	8
I_2	6	6

Para comenzar, vamos a almacenar en el nodo raíz (estado que representa la mesa vacía), la información inicial del problema:

- asignaciones: $\{ \}$
- ganancia actual: 0
- invitados restantes: $\{I_0, I_1, I_2\}$

- porciones restantes: $\{p_0, p_1\}$

Evaluamos la función costo inicial. Queremos sobreestimar la ganancia posible. Para cada porción, sumamos la máxima oferta de cualquier invitado (no tomamos en cuenta ninguna restricción, para así poder encontrar una cota superior)

bound = 0

para la porción 0: $\max(10, 4, 6) = 10$

para la porción 1 P1: $\max(3, 6, 6) = 6$

bound = 10 + 6 = 18

- **paso 1: expandimos el nodo con mayor bound (18)**

Sacamos el nodo raíz del heap, que es el que mayor bound tiene por el momento. Todavía no está completo, así que lo expandimos. Vamos a asignar P0 primero. Invitados restantes: $\{I_0, I_1, I_2\}$. Luego, evaluamos cada posibilidad:

Se selecciona la porción P0 para asignar y se prueban todos los invitados posibles:

Asignar $P_0 \rightarrow I_0$:

- asignaciones: $\{P_0 \rightarrow I_0\}$
- ganancia_actual: 10
- invitados_restantes: $\{I_1, I_2\}$
- porciones_restantes: $\{P_1\}$
- bound: $10 + \max(8, 6) = 18$

Es una asignación válida (no hay vecinos aún así que no hay enemistades por evaluar), se inserta al Heap.

Asignar $P_0 \rightarrow I_1$:

- ganancia_actual: 4
- bound: $4 + \max(3, 6) = 10$
- También válido, se inserta al Heap.

Asignar $P_0 \rightarrow I_2$:

- ganancia_actual: 6
- bound: $6 + \max(3, 8) = 14$
- Válido, se inserta al Heap.

Resulta el estado del Heap tras esta expansión:

Nodo	Bound
$\{P_0 \rightarrow I_0\}$, ganancia 10	18
$\{P_0 \rightarrow I_2\}$, ganancia 6	14
$\{P_0 \rightarrow I_1\}$, ganancia 4	10

A continuación, se selecciona del heap el nodo con el mayor bound nuevamente, que resulta ser el nodo $\{P_0 \rightarrow I_0\}$ con bound 18. Este nodo se expande ahora asignando la porción restante (P1). Los invitados restantes son I1 e I2.

Al evaluar la asignación de P1 a I1, detectamos que I0 e I1 son enemigos y estarían sentados en porciones contiguas (P0 y P1), lo cual hace que esta asignación sea inválida y el nodo es descartado.

En cambio, asignar P1 a I2 no presenta conflicto de enemistades, y se obtiene una ganancia total de $10 + 6 = 16$. Esta es una asignación completa y válida, por lo que se actualiza la mejor solución encontrada hasta el momento. Como se trata de una hoja del árbol, no se inserta nuevamente al heap.

El algoritmo continúa explorando los demás nodos del heap, pero ya con una ganancia objetivo de al menos 16, lo cual permite podar varios caminos que no pueden superar esa ganancia. Por ejemplo, el nodo $\{P0 \rightarrow I2\}$ lleva a dos asignaciones completas con ganancias de 9 y 14, respectivamente, ambas inferiores a la actual mejor solución. Lo mismo ocurre con el nodo $\{P0 \rightarrow I1\}$, cuyas posibles continuaciones tampoco superan la ganancia óptima.

Al finalizar la ejecución, se concluye que la mejor asignación es $\{P0 \rightarrow I0, P1 \rightarrow I2\}$, con una ganancia total de 16.

Esta estrategia de Branch and Bound nos permite reducir significativamente el espacio de búsqueda al evitar expandir nodos que no pueden mejorar la solución actual.

5. Programe ambas propuestas.

La solución de este problema se encuentra en la carpeta adjuntada en la entrega, siendo el archivo ejecutable *subasta_bt.py*. Se adjuntan también dos archivos de prueba: *ofertas.txt* y *enemistades.txt* para poder probar el programa.

Para poder ejecutarlo, correr la siguiente línea:

```
python3 subasta_bt.py ofertas.txt enemistades.txt
```

Siendo el archivo *ofertas.txt* el archivo con la información sobre la cantidad de porciones que el restaurante ofrece, y cuanto ofrece cada invitado por cada porción. El archivo *enemistades.txt* corresponde a la lista de cada invitado sobre con quien no se quiere sentar.

La solución por branch and bound se encuentra adjunta también, siendo el archivo ejecutable *subasta_bb.py*.

Para poder ejecutarlo, correr la siguiente línea:

```
python3 subasta_bb.py ofertas.txt enemistades.txt
```

6. Determine si los programas tienen la misma complejidad que su propuesta teórica.

En cuanto a la propuesta de backtracking, se mantuvo la complejidad de la solución en pseudocódigo. Ambas versiones recorren todas las combinaciones posibles de invitados en las

porciones mediante backtracking, por lo que conservan la complejidad temporal exponencial en función de n y m .

El algoritmo utiliza listas, conjuntos y diccionarios que permiten mantener una implementación eficiente sin alterar la complejidad asintótica del pseudocódigo original. Las listas se usan para representar asignaciones parciales, configuraciones factibles y ofertas por invitado, con operaciones de acceso, copia y rotación en tiempo $O(m)$. Los conjuntos registran invitados ya ubicados y enemigos por persona, permitiendo inserciones y consultas en tiempo promedio $O(1)$. Los diccionarios almacenan ofertas y relaciones de enemistad, también con acceso en $O(1)$. La complejidad temporal general se mantiene exponencial, ya que se exploran todas las combinaciones posibles de asignación en mmm porciones con nnn invitados, y por cada configuración factible se calculan hasta mmm rotaciones, cada una en $O(m^2)$.

En cuanto a la complejidad espacial, se almacenan hasta $O(n^m)$ configuraciones, cada una de tamaño $O(m)$, y las estructuras de entrada utilizan $O(n \cdot m + e)$ espacio, siendo e la cantidad de enemistades.

En cuanto a la propuesta de Branch and Bound, el algoritmo explora todas las asignaciones posibles de m porciones entre n invitados, cuyo número total sin restricciones es $\frac{n!}{(n-m)!}$. Cada nodo del árbol de búsqueda corresponde a una asignación parcial y, en el peor caso (sin poda efectiva), se expanden $O(P(n, m))$ de ellos. Al procesar cada nodo se calcula la cota superior recorriendo las porciones restantes (hasta m) y evaluando las mejores ofertas de los invitados pendientes (hasta n), lo cual cuesta $O(n \cdot m)$, además de comprobar en tiempo constante la validez de la asignación frente a enemistades y realizar inserciones y extracciones en un heap de tamaño $O(\log P(n, m))$ con un costo aproximado de $O(\log P(n, m))$.

La complejidad en el peor caso resulta: $O(\frac{n!}{(n-m)!} \cdot (mn + n \log n))$, de igual forma que la planteada teóricamente.

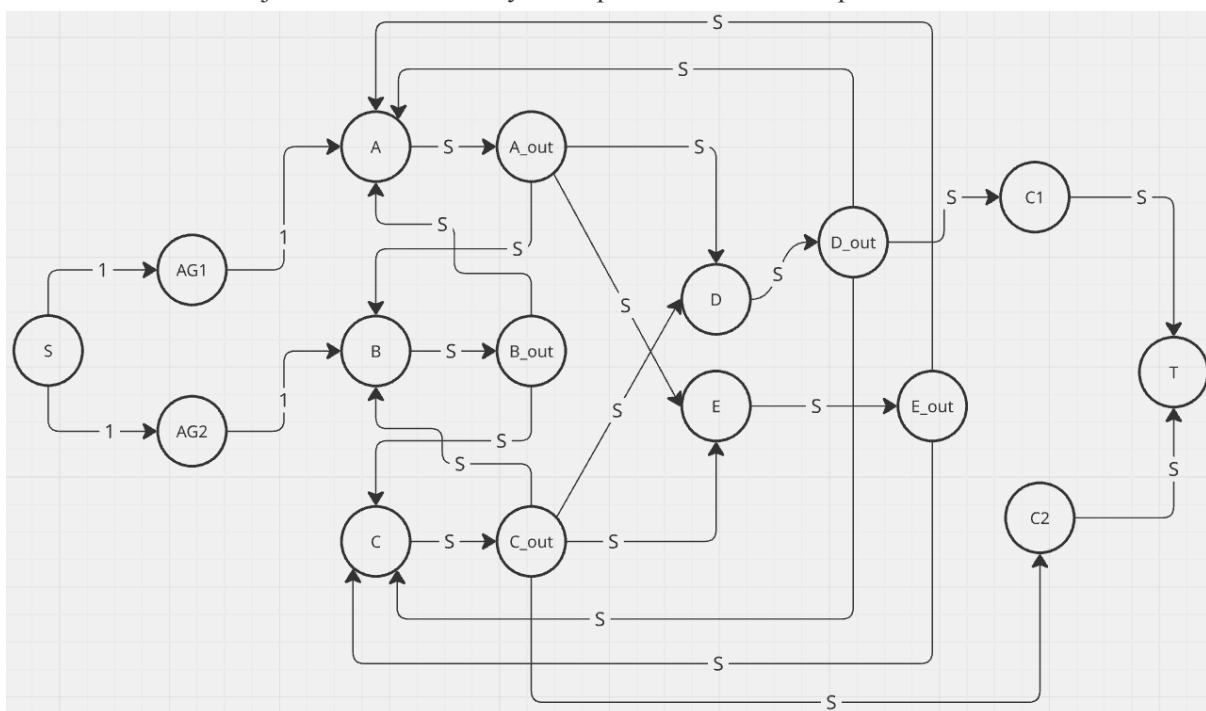
En cuanto al espacio, cada nodo guarda un diccionario de asignaciones (tamaño $\leq m$), dos conjuntos (invitados y porciones restantes, de tamaño $\leq n$ y $\leq m$) y un par de valores escalares, consumiendo $O(n + m)$ por nodo. Dado que el heap puede acumular hasta $O(P(n, m))$ nodos, el uso de memoria en el peor caso es $O(\frac{n!}{(n-m)!} \cdot (n + m))$. La implementación no introduce pasos ni estructuras adicionales que modifiquen estas cotas, por lo que refleja exactamente las mismas limitaciones factoriales del pseudocódigo.

Parte 2: El traslado de Información Secreta

1. Generar una propuesta mediante redes de flujo que solucione el problema. Explicar la idea de esta.

Para la resolución del problema:

1. Se genera un par de nodos por cada ciudad (ci , ci_out) con un eje que parte desde ci hacia ci_out de peso S . Este representa el límite de espías que pueden pasar por esta. Es decir para toda $ci \in Ciudades$ se genera una ci_out con $(ci, ci_out) = S$.
2. Como no podemos tener ejes bi-direccionales, se unirá ci_out con sus respectivos cj adyacentes (es decir con las demás ciudades para las cuales existe un vuelo que las une) y viceversa con $(ci, cj) = S$.
3. Se unirá el nodo fuente con los nodos espías, con valor de eje 1.
4. Se crea un eje que une un nodo espía con su ciudad de origen con peso 1.
5. Se unirá aquellas ciudades centro con el sumidero con valor S .
6. Se creará un eje entre las ciudades y su respectivo centro si es que tiene con valor S .



Al grafo construido a través de ford fulkerson le encontramos su flujo máximo. Y si este flujo es igual a la cantidad de espías que tienen que pasar información, entonces nos indica que es posible con la red que tenemos enviar información y que esta llegue a su destino sin pérdida en el camino (es decir, encontrar para cada espía un camino hacia alguno de los centros).

2. Presentar Pseudocódigo.

```
CONSTRUIR_GRAFO(vuelos, espías, centros, s)
    crear nodo fuente
    crear nodo sumidero
    PARA cada ciudad en vuelos
        agregar nodo ciudad_in y ciudad_out
        agregar arista ciudad_in → ciudad_out con capacidad s
    PARA cada vuelo entre ciudades a y b
        agregar aristas a_out → b_in y b_out → a_in con capacidad s
    PARA cada espía
        agregar arista fuente → espía con capacidad 1
        agregar arista espía → ciudad_in con capacidad 1
    PARA cada centro
        agregar arista ciudad_out → centro
        agregar arista centro → sumidero con capacidad s
    RETORNAR grafo

FORD_FULKERSON(grafo, fuente, sumidero)
    inicializar grafo_residual con las capacidades originales
    flujo_total ← 0
    MIENTRAS exista un camino aumentante desde fuente a sumidero (por BFS)
        encontrar el cuello de botella del camino
        actualizar el grafo residual con el flujo enviado
        sumar el flujo al total
    RETORNAR flujo_total, grafo_residual

ARMADO_DE_RUTA(grafo_original, grafo_residual, nodo, ciudades)
    PARA cada vecino en grafo_original[nodo]
        SI flujo utilizado (original - residual) > 0
            ciudad ← limpiar sufijos del nombre del nodo
            agregar ciudad si no está repetida
            ARMADO_DE_RUTA(grafo_original, grafo_residual, vecino, ciudades)
        salir del bucle

RECONSTRUIR_RUTAS(grafo_original, grafo_residual, espías)
    rutas ← lista vacía
    PARA cada espía
        SI flujo fue asignado a ese espía
            iniciar lista de ciudades
            ARMADO_DE_RUTA(grafo_original, grafo_residual, espía, ciudades)
            agregar ruta formateada con nombre del espía y ciudades
    RETORNAR rutas

TRASLADAR(n, s, archivo_ciudades, archivo_espías)
    vuelos ← leer archivo
    espías, centros ← leer archivo
    grafo ← CONSTRUIR_GRAFO(vuelos, espías, centros, s)
    flujo_max, grafo_residual ← FORD_FULKERSON(grafo, fuente, sumidero)
    SI flujo_max < n
        imprimir "Es imposible lograr el objetivo"
```

SINO

```
rutas ← RECONSTRUIR_RUTAS(grafo, grafo_residual, espías)
imprimir cada ruta
```

3. Realizar un análisis de optimalidad.

En nuestro problema, implementamos la función **FORD_FULKERSON** que, utilizando **BFS** itera buscando caminos de aumento en el grafo residual y finaliza cuando ya no puede encontrar más. Cada incremento de flujo corresponde a trasladar un espía desde su ciudad de origen hasta un centro de investigación respetando las restricciones de capacidad impuestas en las ciudades intermedias. Sabemos que **Ford-Fulkerson**, aplicado con caminos aumentantes, garantiza encontrar el **flujo máximo de manera óptima**.

Teniendo en cuenta que el objetivo del problema es trasladar todos los espías hacia los centros utilizando la red de vuelos y limitando el tránsito por ciudad, podemos concluir que, utilizando **Ford-Fulkerson** y **BFS**, nuestra solución es **óptima**:

- Si el flujo total obtenido es igual al número de espías n , todos los espías lograron ser trasladados exitosamente respetando las restricciones de capacidad.
- Si no es posible alcanzar ese flujo, entonces **no existe ninguna asignación** factible de caminos que cumpla las condiciones planteadas.

Por lo tanto, **la solución obtenida es óptima** bajo las condiciones y restricciones del problema.

4. Realizar análisis de complejidad temporal y espacial. Considere las estructuras de datos que utiliza para llegar a estos.

- n : cantidad de espías (y centros, ya que hay uno por espía)
- v : cantidad de vuelos
- c : cantidad de ciudades (a lo sumo $2v$)

Análisis de Complejidad

Complejidad temporal

1. Construcción del grafo:

Se crean nodos y aristas para representar las ciudades, vuelos, espías y centros.

- **Complejidad:** $O(v + n)$

2. Cálculo del flujo máximo (Ford-Fulkerson):

Se buscan caminos aumentantes mediante BFS.

Como el flujo máximo está acotado por n , y la cantidad de aristas es proporcional a $v + n$,

- **Complejidad:** $O(n * (v + n))$.

3. Armado + Reconstrucción de rutas:

Se reconstruye un camino por espía, recorriendo solo las aristas utilizadas por el flujo.

- **Complejidad:** $O(v + n)$

Complejidad temporal total: $O(n * (v + n))$.

Complejidad espacial

- Se almacenan los grafos original y residual, con nodos y aristas proporcionales a c y $v + n$.
- Además, se almacena una ruta por espía, y en el peor caso, cada ruta puede atravesar hasta $O(c)$ ciudades.

Complejidad espacial total: $O(n * c)$. Como $c \leq 2v$, en el peor caso: $O(n * v)$.

Teniendo como la complejidad total: $O(n * (v + n) + n * v) \approx O(n * (v + n))$.

5. Programar la solución. Incluya la información necesaria para su ejecución. Compare la complejidad de su algoritmo con la del programa.

Ejecucion: python traslado.py cantidad-espías capacidad ciudades.txt espías.txt

Análisis de Complejidad del Algoritmo

- n : cantidad de espías (y centros)
- v : cantidad de vuelos
- c : cantidad de ciudades ($c \leq 2v$)

Complejidad Temporal

1. Lectura de archivos: $O(n + v)$.

2. Construcción del grafo:

Se agregan nodos para cada ciudad $O(c)$, se crean aristas por cada vuelo $O(v)$, y se conectan espías y centros al grafo $O(n)$. Dado que el número de ciudades está acotado por $c < 2v$, esta etapa también tiene una complejidad de $O(n + v)$.

3. Ejecución del algoritmo de Ford-Fulkerson:

Esta fase consiste en iterativamente encontrar caminos aumentantes mediante BFS y aplicarles un aumento de flujo. El número de iteraciones está acotado por el flujo máximo, que es a lo sumo n (uno por espía).

Cada BFS recorre las aristas del grafo, lo cual cuesta $O(n + v)$, y cada aumento (augment) recorre un camino de longitud acotada por $O(c)$, es decir, también $O(v)$.

En consecuencia, esta etapa tiene una complejidad total de $O(n * (v + n))$.

4. Reconstrucción de rutas:

Para cada espía que recibió flujo, se reconstruye su ruta siguiendo las aristas con flujo

utilizado. Como cada arista con flujo se recorre a lo sumo una vez, y las rutas son disjuntas, el total de trabajo en esta etapa no excede $O(n + v)$ DFS.

Por lo tanto, la **complejidad temporal total del programa** es: $O(n * (v + n))$

Complejidad Espacial

En cuanto al espacio, el algoritmo utiliza dos representaciones del grafo: el grafo original y el grafo residual, lo cual requiere espacio proporcional al número de nodos y aristas. Esto representa un costo de $O(n + v)$.

Además, se mantienen estructuras auxiliares como diccionarios de predecesores (camino_aumento), colas para BFS, y estructuras para almacenar las rutas. Cada espía genera una ruta compuesta por ciudades, y en el peor caso, una ruta puede atravesar hasta $O(c)$ ciudades, por lo que el almacenamiento de todas las rutas ocupa $O(n * c)$ espacio.

Dado que $c < 2v$, la **complejidad espacial total** puede expresarse como: $O(n * v)$.

Teniendo como la **complejidad total**: $O(n * (v + n) + n * v) \approx O(n * (v + n))$, la cual concuerda con la del pseudocódigo.

6. ¿Es posible expresar su solución como una reducción polinomial? En caso afirmativo explique cómo y en caso negativo justifique su respuesta.

Para resolver el problema tuvimos que transformar el problema original de traslado de espías en un **problema de flujo máximo en redes**. La transformación realizada consistió en modelar las ciudades, los vuelos y los límites de tránsito mediante un **grafo dirigido con capacidades**.

Formalmente, partimos de un grafo no dirigido $G = (V, E)$, donde V representa ciudades y espías, mientras que los vuelos disponibles se representan mediante las aristas que unen a las ciudades.

La transformación realizada fue:

- Para cada ciudad, se crearon dos nodos, c y c_{out} , conectados mediante una arista con capacidad limitada a S (máximo número de espías que pueden pasar).
- Cada vuelo bidireccional entre ciudades se representó como dos aristas dirigidas: una de $origen_{out}$ a $destino$ y otra de $destino_{out}$ a $origen$, ambas con capacidad S .
- Se conectó la **fuentes** a los nodos espía y los espías a las ciudades de inicio, cada conexión con capacidad 1.
- Se conectaron las ciudades de destino hacia los centros de investigación (en los casos que corresponde), y los centros al **sumidero**, respetando las capacidades establecidas.

Esta transformación se realiza **una sola vez** al comenzar la ejecución del algoritmo, y su costo es proporcional al número de ciudades, vuelos, espías y centros leídos.

Como cada vuelo genera dos aristas dirigidas, y además cada ciudad genera una arista interna, el número total de aristas en el grafo transformado sigue siendo proporcional a la cantidad de elementos de entrada.

Por lo tanto, la transformación de nuestro problema inicial a un problema de flujo máximo en redes se realiza en **tiempo polinomial** $O(m + n + C)$, donde:

- m es el número de vuelos,
- n es el número de espías (y centros),
- c es el número de ciudades.

Luego, la ejecución del algoritmo Ford-Fulkerson con BFS también se mantiene en tiempo polinomial respecto del tamaño del grafo.

Con lo que podemos concluir que nuestra solución puede ser expresada como una reducción polinomial del problema original al problema de flujo máximo, ya que tanto la transformación como la resolución posterior son realizadas en tiempo polinomial.

Parte 3: Las 2 Jornadas de Capacitación

1. Demostrar que, dada una posible solución que nos brindan, se puede fácilmente determinar si se puede cumplir o no con la tarea solicitada.

Una posible solución sería una división de los “ m ” tipos de profesionales en dos subconjuntos y las personas que van a ir un día y el otro:

- Conjunto A: profesiones que van el primer día.
- Conjunto B: profesiones que van el segundo día.
- Personas A: personas que van el primer día.
- Personas B: personas que van el segundo día.

A partir de esa división debemos verificar si se cumple con las restricciones del problema:

Restricción de cantidad máxima por jornada:

Controlamos que tanto para Personas A como para Personas B la cantidad sea menor o igual a p y además que para cada una de esas personas su profesión se encuentre en su respectivo conjunto, además verificamos que cada profesión no se repite en ambos conjuntos.

Restricción de integración por equipo:

Para cada equipo validamos que sus integrantes no se encuentren todos en la misma jornada. Este chequeo puede hacerse en tiempo proporcional al tamaño del equipo. Si hay “ n ” equipos y cada uno tiene a lo sumo “ m ” profesionales, el tiempo total de verificación es $O(n \cdot m)$.

En resumen, dada una solución brindada, podemos verificar si cumple con las condiciones en tiempo polinomial. Por lo tanto, determinar si una solución es válida se puede hacer fácilmente.

2. Demostrar que si desconocemos si es posible lograr una solución, es difícil poder afirmar que existe. Utilizar para eso el problema “Set splitting problem” (suponiendo que sabemos que este es NP-C).

Podemos demostrar que el problema de las 2 jornadas de capacitación es NP-Completo, ya que puede formularse como una instancia del problema conocido como Set Splitting, que es NP-Completo.

En Set Splitting se nos da un conjunto S y una colección de subconjuntos $C_1, \dots, C_n$, de S , y se pregunta si es posible particionar S en dos subconjuntos disjuntos S_1 y S_2 tal que ninguno de los subconjuntos C_i esté completamente contenido en S_1 ni S_2 .

En nuestro caso, los “ m ” tipos de profesionales representan los elementos del conjunto S , y cada uno de los “ n ” equipos de trabajo representan un subconjunto C_i , ya que contiene varios tipos distintos. La restricción de que “ningún equipo debe asistir completo el mismo día” implica que, para cada subconjunto C_i , al menos uno de sus elementos esté asignado a la jornada 1 y los demás a la jornada 2, lo cual es exactamente la condición de Set Splitting.

Por lo tanto, cualquier instancia del problema Set Splitting puede ser reducida en tiempo polinomial al problema de las 2 jornadas, lo que prueba que este último es NP-hard. Además, dado una repartija de los tipos de profesionales en las dos jornadas, podemos ver fácilmente si se cumplen las restricciones, por lo que el problema está también en NP

3. Demostrar que el problema “Set splitting problem” pertenece a NP-C. (Para la demostración se le solicita investigar. La misma se puede realizar partiendo desde “3-SAT” y pasando por “NAE-3-SAT”).

En primer lugar, para demostrar lo solicitado debemos proponer un certificador eficiente a “Set Splitting Problem” (SSP) de modo que se demuestre que el mismo se encuentra en NP. Para ello podemos pensar algo como en el punto 3.1 ya que dada una posible solución a SPP (certificado) y una instancia del mismo buscaremos mediante una secuencia de validaciones saber si el mismo es válido.

- Sea S el conjunto de la instancia que queremos certificar.
- Sea C la colección de subconjuntos de S de la instancia que queremos certificar.
- El certificado consta de dos conjuntos: S_1 y S_2 .
- Validamos que tanto S_1 como S_2 se encuentren contenidos en S .
- Validamos que S_1 y S_2 sean disjuntos.
- Validamos que la unión de S_1 y S_2 sea igual a S .
- Por cada subconjunto C_i en C validamos que no todos sus elementos se encuentren en el mismo subconjunto del certificado.

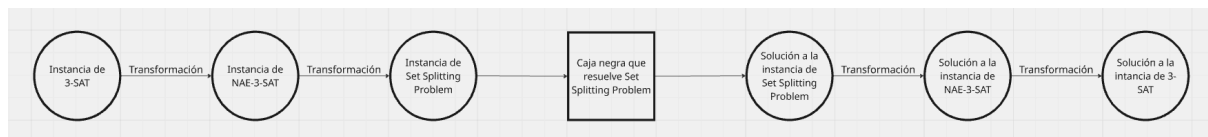
Observamos que la complejidad se compone por:

- $O(2 * (|S|-1)) = O(|S|-1)$ al validar que ambos subconjuntos del certificado se encuentren contenidos en S (restamos 1 porque se espera que tengan menor cardinalidad que S y como mínimo uno de ellos podría tener 1 solo elemento).

- Basta con recorrer los elementos de uno de ellos y comparar contra los elementos del otro, por ende podemos acotar por $O(|S|^2)$ al validar que S_1 y S_2 sean disjuntos.
- La complejidad de $S_1 \cup S_2 = S$ equivale a $O(|S|)$ por recorrer los elementos de S y comparar (en tiempo constante) la inclusión de cada elemento en $S_1 \cup S_2$.
- Por cada elemento en cada C_i validamos que el mismo se encuentre en uno de los dos subconjuntos del certificado en $O(|C|)$ recordemos que comprobar la inclusión de un elemento en un conjunto es $O(1)$.
- Finalmente nuestro verificador eficiente ejecuta en $O(|S| + |C|)$.

Posteriormente podemos demostrar que el problema Set Splitting es NP-Completo tras demostrar que también pertenece a NP-Hard mediante una secuencia de reducciones polinómicas desde problemas ya conocidos. El camino que seguiremos es:
De 3-SAT a NAE-3-SAT y luego a Set Splitting.

Por lo tanto seguiremos el siguiente camino:



Queremos demostrar que $3\text{-SAT} \leq_p \text{NAE-3-SAT} \leq_p \text{Set Splitting Problem}$.

Vamos de a poco:

1- Sabemos que 3-SAT es NP-completo:

Este problema es uno de los más conocidos. En 3-SAT se nos da una fórmula lógica con cláusulas, donde cada cláusula tiene exactamente tres literales o sus negaciones. El objetivo es ver si existe una forma de asignar valores verdaderos o falsos a estos literales, de modo que toda la fórmula quede verdadera.

2- Reducción de 3-SAT a NAE-3-SAT:

NAE-3-SAT (Not-All-Equal) es una variante de 3-SAT, pero con una condición distinta: además de que cada cláusula tiene tres literales, queremos que no todos tengan el mismo valor. Es decir, en cada cláusula no todos los literales deben ser verdaderos y no todos los literales deben ser falsos, esto implica que en cada cláusula debe haber al menos un literal verdadero y al menos uno falso. Para forzar a que esto suceda en cada cláusula debemos transformar la instancia de 3-SAT en una instancia de NAE-3-SAT. Para esto recordemos que las cláusulas de una instancia de 3-SAT son de la forma $(x \text{ or } y \text{ or } z)$ por lo tanto nuestra transformación consistirá en agregar un nuevo literal α resultando en dos cláusulas a partir de la anterior, por lo tanto quedarían de la siguiente forma $(x \text{ or } y \text{ or } \alpha)$ and $(z \text{ or } \alpha \text{ or } !\alpha)$. La complejidad de esta transformación es $O(|C|)$ dado que repetimos esta operación por cada cláusula C_i de la instancia original de 3-SAT.

De la forma detallada en el párrafo anterior se demuestra que NAE-3-SAT es NP-Hard, resta demostrar que también se encuentra en NP. Para ello necesitamos proponer un certificador eficiente de NAE-3-SAT que ejecute en tiempo polinomial, el mismo tomará una instancia de NAE-3-SAT y como

certificador un diccionario donde las claves serán los literales y sus valores asociados en él, el valor que deberán tomar para que la instancia resulte verdadera. La idea es que el mismo se base en iterar las cláusulas de la instancia y por cada una corrobore que cada literal se encuentre en el certificador además de contar aquellos verdaderos y falsos a partir de sus valores en el diccionario. Si no existen al menos un literal con valor 0 y otro con valor 1 en la cláusula (osea todos tienen el mismo valor) entonces devuelve directamente “falso”, esto se repite por cada cláusula hasta haber corroborado todas. Si mientras evalúa una cláusula no se encuentra un literal en el diccionario entonces también devuelve “falso”. Una vez haya recorrido todas las cláusulas validando que se cumpla que para cada una de ellas no se repite el mismo valor en todos sus literales, entonces devuelve “verdadero”. La complejidad del mismo es entonces $O(|C|)$ con C la cantidad de cláusulas de la instancia, tengamos en cuenta que en cada cláusula se verifica la entrada de tres literales en el certificado en tiempo constante. Hemos demostrado que NAE-3-SAT se encuentra también en NP.

3. Reducción de NAE-3-SAT a Set Splitting:

Ahora vamos a transformar una instancia de NAE-3-SAT en una instancia del problema Set Splitting (conjunto S y la colección C de subconjuntos de S). Aquí el conjunto S se compone por los literales que se encuentran en las cláusulas (y sus negaciones) mientras que para cada cláusula (que contiene tres literales), construimos un subconjunto con esos tres elementos. Por ejemplo, si una cláusula es $(x \vee -y \vee z)$, entonces en el problema Set Splitting armamos el subconjunto $[x, -y, z]$. De esta forma cada cláusula equivale a un C_i de la colección C . También agregaremos unos subconjuntos a C que constan de pares de literales de la forma $\{x, !x\}$ ya que de esta forma incluimos una restricción adicional que impide que x y $-x$ estén en el mismo conjunto en la solución, ya que deben representar valores opuestos (por ejemplo, si x está en el conjunto S_1 , entonces $-x$ debe estar en S_2 , y al revés). Por lo tanto la complejidad es $O(|C|)$ ya que recorreremos las cláusulas de la instancia de NAE-3-SAT para construir S y C de la instancia de Set Splitting.

Para demostrar que Set Splitting se encuentra en NP debemos dar un certificador eficiente, el mismo tomará por parámetros la instancia del problema (el conjunto S y la colección C de subconjuntos del mismo) y un certificador que consta de dos conjuntos (S_1, S_2).

- 1) Validará que cada elemento tanto de S_1 como de S_2 se encuentre en S (se espera que S_1 y S_2 se encuentren contenidos en S y por ende acotamos esta validación con $O(|S|)$).
- 2) Validará que S_1 y S_2 sean disjuntos (nuevamente $O(|S|)$ por lo mencionado en 1).
- 3) Validará que la unión de S_1 y S_2 sea igual a S (validamos que dicha unión sea igual a S y entonces $O(|S|)$).
- 4) Validará por cada C_i de C no suceda que todos los elementos de C_i pertenecen completamente a S_1 ni a S_2 (pero que pertenece a alguno).

Para determinar la complejidad debemos entonces validar la integridad de S_1 y S_2 del certificado además de validar que en cada cláusula se cumpla la condición de que no todos sus elementos se encuentren en el mismo subconjunto solución, como no sabemos qué cantidad de elementos puede tener cada cláusula acotamos por $O(|S| * |C|)$, demostrando entonces que existe un certificador eficiente de modo que Set Splitting pertenece a NP.

4. Transformación de la solución de Set Splitting a la solución de la instancia construida de NAE-3-SAT:

El objetivo en Set Splitting es dividir el conjunto S en dos grupos S_1 y S_2 , de modo que ninguno de los subconjuntos que armamos quede totalmente dentro de uno solo de esos dos grupos.

Eso significa que en cada uno de esos subconjuntos, que equivalen a cláusulas de NAE-3-SAT (sin considerar aquellas compuestas por pares de literales) al menos un literal tiene que estar en S_1 y otro en S_2 , lo cual refleja exactamente la condición de NAE: no todos iguales, tiene que haber al menos una verdadera y una falsa (aquellas verdaderas pertenecen a S_1 en la solución a Set Splitting y las falsas a S_2). Entonces, si logramos encontrar esa división de S , también estamos resolviendo la fórmula de NAE-3-SAT. La complejidad de esta transformación se basa en recorrer S_1 y S_2 para saber qué valores deberán tener los literales en nuestra instancia de NAE-3-SAT para resolverla con éxito, con lo cual la misma es $O(|S|)$.

5. Transformación de la solución de NAE-3-SAT la solución de nuestra instancia original de 3-SAT:

Los valores de los literales en la solución de la instancia construida para NAE-3-SAT corresponden a una solución de la instancia original de 3-SAT de la cual partimos. Recordemos que “separamos” las cláusulas originales para poder aplicar la restricción de “no todos iguales” la cual ahora no tenemos, por lo tanto los mismos valores de los literales que solucionan la instancia construida de NAE-3-SAT también solucionan la instancia original de 3-SAT.

Por lo tanto hemos demostrado que Set Splitting Problem pertenece a NP y a NP-HARD, por lo tanto también pertenece a NP-C.

4. Una persona afirma tener un método eficiente para responder si es posible o no cualquiera sea la instancia. Utilizando el concepto de transitividad y la definición de NP-C explique qué ocurriría si se demuestra que la afirmación es correcta.

Si una persona afirma tener un método eficiente para responder si es posible o no cualquiera sea la instancia, esto significaría que el problema se puede resolver en tiempo polinomial. Habiendo demostrado que P2J es NP-Completo sabemos que el mismo se encuentra en NP-Hard y NP, dicho esto, por definición de NP-Hard sabemos que cualquier problema en NP puede reducirse en tiempo polinomial a este (además si existe tal algoritmo entonces P2J también estaría en P). Si transformamos una instancia de un problema X en NP a una instancia de P2J en tiempo polinomial y la resolvemos mediante el supuesto algoritmo eficiente, la solución a la instancia resultante de P2J tras ser transformada en tiempo polinomial equivale a la solución de la instancia del problema X . De esta forma todos los problemas en NP podrían resolverse en tiempo polinómico por transitividad, significando que $P = NP$, resolviendo uno de los problemas del milenio.

5. Un tercer problema al que llamaremos X se puede reducir polinomialmente al problema de “Las 2 jornadas de capacitación”, qué podemos decir acerca de su complejidad?

Si un problema X se reduce en tiempo polinomial a P2J, entonces podemos concluir que P2J es al menos tan difícil que X . Es decir, una solución eficiente para el problema de las 2 jornadas permitiría resolver cualquier instancia del problema X mediante esa reducción. Pero esta relación no alcanza para afirmar que el problema de las 2 jornadas sea NP-Hard o NP-Completo, a menos que

sepamos además que X pertenece a una de esas clases. Entonces la complejidad del problema de las 2 jornadas dependerá de la complejidad de X : si X es NP-Hard, entonces el problema de las 2 jornadas también lo será y si no, no se puede concluir nada con certeza sólo a partir de esta reducción

Correcciones

Parte 1

Función costo - Branch and Bound

Se corrige un **error de tipeo** en la siguiente sección de texto:

Se implementa una función **costo** que estima de forma optimista la mejor ganancia posible a partir de un estado parcial. A continuación, se detalla un ejemplo para la función de costo planteada.

$$\text{Siendo } bound(A) = g(A) + \sum_{p \in \text{porciones restantes}} \max oferta_{i,p}$$

con $g(A)$ ganancia de la asignación actual, e $i \in \text{invitados no sentados}$

Por ejemplo: tenemos $m = 2$, invitados $I = [I1, I2, I3]$

invitado/número de porción	0	1
Elon (I1)	10	3
Steve (I2)	4	8
Mark (I3)	6	6

$bound = 0$

para la porción 0: $\max(10, 4, 6) = 10$

para la porción 1 P1: $\max(3, 8, 6) = 8$

$bound = 10 + 8 = 18$

Asignamos este bound al nodo raíz, para arrancar.

Pseudocódigo - backtracking

Se agrega en el pseudocódigo presentado para backtracking la consideración de dejar un asiento vacío.

```
// E: Diccionario donde E[i] es el conjunto de invitados con los que i está enemistado.
// P: Diccionario donde P[i][j] es la oferta del invitado i por la porción j.
// n: Número de invitados.
// m: Número de porciones (n > m).
```

```
// Retorna la asignación de invitados a porciones que maximiza la ganancia.

UbicacionesFactibles(invitados, E, P, n, m)
    uFactibles = []
    ObtenerUbicaciones(invitados, E, [], [], 0, n, m, uFactibles)
    Retornar MaximaGananciaConRotaciones(uFactibles, P, m)

ObtenerUbicaciones(invitados, E, uParcial, i, n, m, uFactibles)
    Si len(uParcial) == m
        Si EsFactibleCircular(uParcial, E)
            uFactibles.append(uParcial.copy())
            Retornar

    Si i >= n
        // Caso base: ya se consideraron todos los invitados para esta posición,
        // agregamos la opción de dejar asiento vacío
        uParcial.append(None)
        ObtenerUbicaciones(invitados, E, uParcial, invitadosUsados, i+1, n, m,
uFactibles)
        uParcial.pop()
        Retornar

    Para cada invitado desde i hasta n-1
        Si invitado no está en invitadosUsados Y PuedoUbicarlo(uParcial,
invitados[invitado], E)
            uParcial.append(invitados[invitado])
            agregar invitados[invitado] a invitadosUsados
            ObtenerUbicaciones(invitados, E, uParcial, invitadosUsados, i+1, n, m,
uFactibles)
            uParcial.pop()
            eliminar invitados[invitado] de invitadosUsados

    // Además, consideramos dejar vacío este asiento
    uParcial.append(None)
    ObtenerUbicaciones(invitados, E, uParcial, invitadosUsados, i+1, n, m, uFactibles)
    uParcial.pop()

PuedoUbicarlo(uParcial, invitadoActual, E)
    Si len(uParcial) == 0
        Retornar Verdadero

    invitadoAnterior = uParcial[-1]
    Si invitadoActual está en E[invitadoAnterior] O invitadoAnterior está en
E[invitadoActual]
        Retornar Falso

    Retornar Verdadero
```

```

EsFactibleCircular(uParcial, E)
    primerInvitado = uParcial[0]
    ultimoInvitado = uParcial[-1]

    Si primerInvitado == None O ultimoInvitado == None
        Retornar Verdadero
    Si ultimoInvitado está en E[primerInvitado]
        Retornar Falso
    Retornar Verdadero

MaximaGananciaConRotaciones(uFactibles, P, m)
    gananciaMaxima = 0
    asignacionOptima = []

    Para cada ubicacion en uFactibles
        Para i desde 0 hasta m-1
            rotacion = Rotar(ubicacion, i)
            ganancia = 0
            Para j desde 0 hasta m-1
                invitado = rotacion[j]
                Si invitado != None
                    ganancia += P[invitado][j]
            Si ganancia > gananciaMaxima
                gananciaMaxima = ganancia
                asignacionOptima = rotacion

    Retornar (gananciaMaxima, asignacionOptima)

Rotar(ubicacion, k)
    n = len(ubicacion)
    Retornar ubicacion[k:n] + ubicacion[0:k]

```

Pseudocódigo - Branch & Bound

Se agrega en el pseudocódigo presentado para branch & bound la consideración de dejar un asiento vacío.

```

BranchAndBound(m, n, ofertas, enemistades):
    mejor_ganancia = 0
    mejor_asignacion = []

    nodo_raiz = CrearNodo(
        asignaciones = [],
        ganancia_actual = 0,
        invitados_restantes = {0..n-1},
        porciones_restantes = {0..m-1},
        bound = costo([], 0, {0..n-1}, {0..m-1}, ofertas)
    )

```

```
asignacion = MaxHeap()
insertar(asignacion, nodo_raiz)

mientras no esta_vacio(asignacion):
    nodo ← extraer_max(asignacion)

    si nodo.bound ≤ mejor_ganancia:
        continuar # podemos

    si size(nodo.porciones_restantes) == 0:
        si nodo.ganancia_actual > mejor_ganancia:
            mejor_ganancia ← nodo.ganancia_actual
            mejor_asignacion ← nodo.asignaciones
        continuar

    porción_actual ← seleccionar_porcion(nodo.porciones_restantes)

    #Caso de dejar la porción vacía
    nueva_asignacion ← copiar(nodo.asignaciones)
    nueva_asignacion[porción_actual] ← None # representa asiento vacío

    nuevos_invitados ← nodo.invitados_restantes # ningún invitado asignado
    nuevas_porciones ← nodo.porciones_restantes - {porción_actual}

    nuevo_bound ← costo(nueva_asignacion, nodo.ganancia_actual, nuevos_invitados,
nuevas_porciones, ofertas)

    nuevo_nodo ← CrearNodo(
        asignaciones = nueva_asignacion,
        ganancia_actual = nodo.ganancia_actual,
        invitados_restantes = nuevos_invitados,
        porciones_restantes = nuevas_porciones,
        bound = nuevo_bound
    )

    insertar(asignacion, nuevo_nodo)

    #Caso asignamos un invitado
    para cada invitado en nodo.invitados_restantes:
        si es_valida_asignacion(nodo.asignaciones, porción_actual, invitado,
enemistades, m):
            nueva_asignación ← copiar(nodo.asignaciones)
            nueva_asignación[porción_actual] ← invitado

            nueva_ganancia ← nodo.ganancia_actual +
ofertas[invitado][porción_actual]
```

```
nuevos_invitados ← nodo.invitados_restantes - {invitado}
nuevas_porciones ← nodo.porciones_restantes - {porción_actual}

nuevo_bound ← costo(nueva_asignación, nueva_ganancia, nuevos_invitados,
nuevas_porciones, ofertas)

nuevo_nodo ← CrearNodo(
    asignaciones = nueva_asignación,
    ganancia_actual = nueva_ganancia,
    invitados_restantes = nuevos_invitados,
    porciones_restantes = nuevas_porciones,
    bound = nuevo_bound
)

insertar(heap, nuevo_nodo)

retornar mejor_ganancia, mejor_asignacion

func costo(asignaciones, ganancia_actual, invitados_restantes, porciones_restantes,
ofertas):
    bound ← ganancia_actual

    para cada porción p en porciones_restantes:
        mejor_oferta ← 0
        para cada invitado i en invitados_restantes:
            mejor_oferta ← max(mejor_oferta, ofertas[i][p])
        bound ← bound + mejor_oferta

    retornar bound

func es_valida_asignacion(asignaciones, porcion_actual, invitado, enemistades, m):
    vecinos ← {
        (porcion_actual - 1 + m) mod m,
        (porcion_actual + 1) mod m
    }

    para cada vecino en vecinos:
        si vecino ∈ asignaciones:
            invitado_vecino ← asignaciones[vecino]
            si invitado_vecino == None:
                continuar # porción vacía
            si (invitado, invitado_vecino) ∈ enemistades o (invitado_vecino, invitado)
            ∈ enemistades:
                retornar falso

    retornar verdadero
```


Ejemplo paso a paso - Branch & Bound

Se agrega al ejemplo paso a paso provisto, el caso en el que se deja un asiento vacío.

Paso a Paso de solución por Branch and Bound

Supongamos que tenemos el siguiente problema de ejemplo:

- Porciones $m = 2$ (p_0, p_1)
- Invitados $n = 3$ (I_0, I_1, I_2). Sabemos que I_0 no quiere sentarse al lado de I_1 .

Los invitados realizan las siguientes ofertas:

invitado/número de porción	p_0	p_1
I_0	10	3
I_1	4	8
I_2	6	6

Para comenzar, vamos a almacenar en el nodo raíz (estado que representa la mesa vacía), la información inicial del problema:

- asignaciones: $\{ \}$
- ganancia actual: 0
- invitados restantes: $\{I_0, I_1, I_2\}$
- porciones restantes: $\{p_0, p_1\}$

Evaluamos la función costo inicial. Queremos sobreestimar la ganancia posible. Para cada porción, sumamos la máxima oferta de cualquier invitado (no tomamos en cuenta ninguna restricción, para así poder encontrar una cota superior)

$\text{bound} = 0$

para la porción 0: $\max(10, 4, 6) = 10$

para la porción 1 P_1 : $\max(3, 8, 6) = 6$

$\text{bound} = 10 + 8 = 18$

Se inserta el nodo raíz al heap con bound 18.

- **paso 1: expandimos el nodo con mayor bound (18)**

Sacamos el nodo raíz del heap, que es el que mayor bound tiene por el momento. Todavía no está completo, así que lo expandimos. Vamos a asignar P_0 primero. Invitados restantes: $\{I_0, I_1, I_2\}$ más la opción de dejar el asiento vacío. Luego, evaluamos cada posibilidad:

Se selecciona la porción P_0 para asignar y se prueban todos los invitados posibles:

Asignar $P_0 \rightarrow I_0$:

- asignaciones: $\{P_0 \rightarrow I_0\}$
- ganancia_actual: 10

- invitados_restantes: {I1, I2}
- porciones_restantes: {P1}
- bound: $10 + \max(8, 6) = 18$

Es una asignación válida (no hay vecinos aún así que no hay enemistades por evaluar), se inserta al Heap.

Asignar P0 → I1:

- ganancia_actual: 4
- bound: $4 + \max(3, 6) = 10$
- También válido, se inserta al Heap.

Asignar P0 → I2:

- ganancia_actual: 6
- bound: $6 + \max(3, 8) = 14$
- Válido, se inserta al Heap.

Asignar P0 → VACIO:

- asignaciones: {P0 → VACIO}
- ganancia_actual: 0
- invitados_restantes: {I0, I1, I2}
- porciones_restantes: {P1}
- bound: $0 + \max(3, 8, 6) = 8$
- Válido, se inserta al Heap.

Resulta el estado del Heap tras esta expansión:

Nodo	Bound
{P0 → I0}, ganancia 10	18
{P0 → I2}, ganancia 6	14
{P0 → I1}, ganancia 4	10
{P0 → VACIO}, ganancia 0	8

A continuación, se selecciona del heap el nodo con el mayor bound nuevamente, que resulta ser el nodo {P0 → I0} con bound 18. Este nodo se expande ahora asignando la porción restante (P1). Los invitados restantes son I1 e I2.

Al evaluar la asignación de P1 a I1, detectamos que I0 e I1 son enemigos y estarían sentados en porciones contiguas (P0 y P1), lo cual hace que esta asignación sea inválida y el nodo es descartado.

En cambio, asignar P1 a I2 no presenta conflicto de enemistades, y se obtiene una ganancia total de $10 + 6 = 16$. Esta es una asignación completa y válida, por lo que se actualiza la mejor solución encontrada hasta el momento. Como se trata de una hoja del árbol, no se inserta nuevamente al heap.

Al evaluar dejar P1 vacío, llegamos a una ganancia máxima de 10, pero no supera la mejor solución encontrada (asignar P1 a I2), así que se descarta.

El algoritmo continúa explorando los demás nodos del heap, pero ya con una ganancia objetivo de al menos 16, lo cual permite podar varios caminos que no pueden superar esa ganancia. Por ejemplo, el nodo $\{P0 \rightarrow I2\}$ lleva a dos asignaciones completas con ganancias de 9 y 14, respectivamente, ambas inferiores a la actual mejor solución. Lo mismo ocurre con el nodo $\{P0 \rightarrow I1\}$, cuyas posibles continuaciones tampoco superan la ganancia óptima.

Al finalizar la ejecución, se concluye que la mejor asignación es $\{P0 \rightarrow I0, P1 \rightarrow I2\}$, con una ganancia total de 16.

Esta estrategia de Branch and Bound nos permite reducir significativamente el espacio de búsqueda al evitar expandir nodos que no pueden mejorar la solución actual.

Corrección en el programa desarrollado - Branch & Bound

Se corrigió el código desarrollado para la propuesta de Branch & Bound tal que pase las pruebas de la cátedra. Se incorporó en el código un arreglo en la consideración de asientos vacíos. Ahora funciona correctamente y elige la opción con mayor ganancia teniendo en cuenta que esta opción puede implicar dejar asientos vacíos.

Los programas se continúan corriendo de la misma manera:

```
python3 subasta_bb.py ofertas.txt enemistades.txt
```

Análisis de complejidad real

Se añade al análisis de complejidad realizado la consideración de la complejidad de las estructuras del lenguaje Python utilizadas.

Branch & Bound

En cuanto a la propuesta de Branch and Bound, el algoritmo explora todas las asignaciones posibles de m porciones entre n invitados, cuyo número total sin restricciones es $\frac{n!}{(n-m)!}$. Cada nodo del árbol de búsqueda corresponde a una asignación parcial y, en el peor caso (sin poda efectiva), se expanden $O(P(n,m))$ de ellos. Al procesar cada nodo se calcula la cota superior recorriendo las porciones restantes (hasta m) y evaluando las mejores ofertas de los invitados pendientes (hasta n), lo cual cuesta $O(n * m)$.

En el programa desarrollado, acceder al valor `ofertas[i][p]` (siendo `ofertas` un dict de dicts), tiene un costo amortizado de $O(1)$ por acceso. La validez de asignación frente a enemistades también se hace en tiempo constante, gracias a que se usa un set S para las enemistades `enemistades[invitado]`, y acceso directo con `in`.

Se utiliza un heap (MaxHeap) para mantener una cola de prioridad. En Python, `heapq` implementa un heap binario min-heap, por lo que para simular un max-heap se invierte el valor de prioridad (se usa `-bound`).

Realizar inserciones y extracciones del heap de tamaño $O(\log P(n, m))$ tiene un costo aproximado de $O(\log P(n, m))$

La complejidad en el peor caso resulta: $O\left(\frac{n!}{(n-m)!}\right) * (mn + n \log n)$, de igual forma que la planteada teóricamente.

En cuanto al espacio, cada nodo guarda un diccionario de asignaciones (tamaño $\leq m$), dos conjuntos (invitados y porciones restantes, de tamaño $\leq n$ y $\leq m$) y un par de valores escalares (ganancia actual y bound), consumiendo $O(n + m)$ por nodo.

Dado que el heap puede acumular hasta $O(P(n, m))$ nodos, el uso de memoria en el peor caso es $O\left(\frac{n!}{(n-m)!}\right) * (n + m)$. La implementación no introduce pasos ni estructuras adicionales que modifiquen estas cotas, por lo que refleja exactamente las mismas limitaciones factoriales del pseudocódigo.

Parte 3

Pregunta 1 - Demostrar que, dada una posible solución que nos brindan, se puede fácilmente determinar si se puede cumplir o no con la tarea solicitada.

Una posible solución S puede estar compuesta por los conjuntos $P1$ y $P2$ ($S = \{P1, P2\}$) donde tanto $P1$ como $P2$ poseen integrantes del personal y cada uno de ellos tiene una profesión.

Restricción de cantidad máxima por jornada:

Controlamos que tanto para $P1$ como para $P2$ la cantidad de elementos que tenga (osea la cantidad de personal con la que cuentan) sea menor o igual a p .

Restricción de tipos de profesión no repetidas en ambas jornadas:

Por otro lado debemos validar que al evaluar una persona de un conjunto P_i , su profesión no la tenga una persona del otro conjunto (si primero guardamos las profesiones del personal de $P1$ en un conjunto A y luego a medida que evaluamos el personal de $P2$ guardamos sus profesiones en un conjunto B se trata de una operación $O(1)$ validar que la profesión de la persona evaluada actualmente no está en A). También debemos validar que una persona no se encuentre repetida en $P1$ y en $P2$ (aunque si lo está también violaría la restricción de que las profesiones no se repitan en ambas jornadas, esta validación la podemos hacer en tiempo constante por ser $P1$ y $P2$ conjuntos).

Restricción de integración por equipo:

Para cada equipo validamos que sus integrantes no se encuentren todos en la misma jornada. Este chequeo puede hacerse en tiempo proporcional al tamaño del equipo. Si hay “ n ” equipos y cada uno tiene a lo sumo “ m ” profesionales, el tiempo total de verificación es $O(n \cdot m)$.

En resumen, dada una solución brindada de la forma indicada, podemos verificar si cumple

con las condiciones en tiempo polinomial. Por lo tanto, determinar si una solución es válida se puede realizar de manera eficiente.

Pregunta 2 - Demostrar que si desconocemos si es posible lograr una solución, es difícil poder afirmar que existe. Utilizar para eso el problema “Set splitting problem” (suponiendo que sabemos que este es NP-C).

Vamos a demostrar que el problema de las 2 jornadas de capacitación es NP-completo (demostrando que es NP y NP-hard), mediante una reducción polinomial desde el problema Set Splitting, que se sabe que es NP-completo (y por lo tanto es NP-hard). El objetivo es tomar una instancia del Set Splitting y construir, en tiempo polinomial, una instancia del problema de las 2 jornadas tal que ambas tengan la misma respuesta (sí o no). Esto nos permitirá probar que el problema de las 2 jornadas es al menos tan difícil como Set Splitting.

Recordemos que en Set Splitting se nos da un conjunto S y una colección de subconjuntos $C_1, C_2, \dots, C_n$ de S . El problema consiste en decidir si es posible dividir S en dos subconjuntos disjuntos S_1 y S_2 , de manera que ningún subconjunto C_i esté completamente contenido ni en S_1 ni en S_2 .

Para construir la instancia equivalente del problema de las 2 jornadas, identificamos los m tipos de profesionales con los elementos del conjunto S , y los n equipos de trabajo con los subconjuntos C_i . Cada equipo está formado por profesionales de distintos tipos. La restricción del problema de las 2 jornadas indica que ningún equipo debe asistir completo el mismo día, lo cual equivale a decir que para cada conjunto C_i , al menos uno de sus miembros debe estar asignado a una jornada y los demás a la otra. Esta es la misma condición que exige el Set Splitting Problem.

Como la transformación desde una instancia de Set Splitting a una instancia del problema de las 2 jornadas puede hacerse en tiempo polinomial ($O(m)$ siendo m la cantidad de elementos de S que para P2J representan los tipos de profesionales más $O(n)$ siendo n la cantidad de subconjuntos C_i y para P2J representa los equipos de trabajo). Resolver la instancia del problema de las 2 jornadas que resulta de la transformación indicada permite saber si existe solución para la instancia inicial de Set Splitting (si la ejecución de P2J devuelve “sí” entonces podemos identificar los tipos de profesión de cada jornada en $O(m)$ y cada jornada representa S_1 y S_2 para la solución de SSP). Concluimos que el problema de las 2 jornadas es entonces NP-hard. Además, dados dos conjuntos de personas, dado que cada una tiene un tipo de profesión, es sencillo verificar mediante un algoritmo eficiente como el mencionado en el punto 1 si las restricciones se cumplen, lo que significa que el problema es NP. Finalmente, hemos demostrado que el problema de las 2 jornadas es NP y NP-hard, en consecuencia es NP-completo.

Pregunta 3 - parte 2: Reducción de 3-SAT a NAE-3-SAT

Agregaremos un paso a la reducción, de modo que nuestra cadena de reducciones será la siguiente:

Queremos demostrar que $3\text{-SAT} \leq_p \text{NAE-4-SAT} \leq_p \text{NAE-3-SAT} \leq_p \text{Set Splitting Problem}$.

Vamos de a poco:

1- Sabemos que 3-SAT es NP-completo:

Este problema es uno de los más conocidos. En 3-SAT se nos da una fórmula lógica con cláusulas, donde cada cláusula tiene exactamente tres literales o sus negaciones. El objetivo es ver si existe una forma de asignar valores verdaderos o falsos a estos literales, de modo que toda la fórmula quede verdadera.

2- Reducción de 3-SAT a NAE-4-SAT:

NAE-4-SAT (Not-All-Equal) es una variante de 3-SAT, pero con dos modificaciones: cada cláusula tiene cuatro literales y queremos que no todos tengan el mismo valor. Es decir, en cada cláusula no todos los literales deben ser verdaderos y no todos los literales deben ser falsos, esto implica que en cada cláusula debe haber al menos un literal verdadero y al menos uno falso. Para forzar a que esto suceda en cada cláusula debemos transformar la instancia de 3-SAT en una instancia de NAE-4-SAT. Para esto recordemos que las cláusulas de una instancia de 3-SAT son de la forma $(x \text{ or } y \text{ or } z)$ por lo tanto nuestra transformación consistirá en agregar un nuevo literal α por cada cláusula resultando en dos nuevas cláusulas de la siguiente forma $(x \text{ or } y \text{ or } z \text{ or } \alpha)$ and $(x \text{ or } y \text{ or } z \text{ or } !\alpha)$. Realizamos esta creación de dos cláusulas nuevas para la instancia de NAE-4-SAT ya que necesitamos que al menos uno de los literales de la cláusula original tenga como valor TRUE dado que una solución sin contemplar la segunda cláusula es $x = y = z = 0, \alpha = 1$ lo cual para la instancia original de 3-SAT no equivaldría a una solución correcta. De esta forma forzamos a que alguno de los literales de la instancia original tome como valor TRUE.

La complejidad de esta transformación es $O(|C|)$ dado que repetimos esta operación por cada cláusula C_i de la instancia original de 3-SAT (siendo C el conjunto de cláusulas de la instancia de 3-SAT).

De la forma detallada en el párrafo anterior se demuestra que NAE-4-SAT es NP-Hard, resta demostrar que también se encuentra en NP. Para ello necesitamos proponer un certificador eficiente de NAE-4-SAT que ejecute en tiempo polinomial, el mismo tomará una instancia de NAE-4-SAT y como certificador un diccionario donde las claves serán los literales y sus valores asociados en él, el valor que deberán tomar para que la instancia resulte verdadera. La idea es que el mismo se base en iterar las cláusulas de la instancia y por cada una corrobore que cada literal se encuentre en el certificador además de contar aquellos verdaderos y falsos a partir de sus valores en el diccionario. Si no existen al menos un literal con valor 0 y otro con valor 1 en la cláusula (osea todos tienen el mismo valor) entonces devuelve directamente “falso”, esto se repite por cada cláusula hasta haber corroborado todas. Si mientras evalúa una cláusula no se encuentra un literal en el diccionario entonces también devuelve “falso”. Una vez haya recorrido todas la cláusulas validando que se cumpla que para cada una de ellas no se repite el mismo valor en todos sus literales, entonces devuelve “verdadero”. La complejidad del mismo es entonces $O(C)$ con C la cantidad de cláusulas de la instancia, tengamos en cuenta que en cada cláusula se verifica la entrada de cuatro literales en el certificado en tiempo constante.

Finalmente hemos demostrado que NAE-4-SAT se encuentra en NP y NP-Hard, por lo tanto también en NP-C.

3. Reducción de NAE-4-SAT a NAE-3-SAT:

NAE-3-SAT (Not-All-Equal) es una variante de NAE-4-SAT donde cada cláusula tiene tres literales (se preserva la idea de que queremos que no todos tengan el mismo valor). Dada una instancia de NAE-4-SAT sabemos que sus cláusulas son de la forma $(x \text{ or } y \text{ or } z \text{ or } \alpha)$. Queremos que cada cláusula tenga ahora tres literales, para ello introduciremos dos nuevos literales β , γ dividiendo cada cláusula de la instancia de NAE-4-SAT en tres cláusulas de NAE-3-SAT de la siguiente manera:

$$(x \text{ or } y \text{ or } z \text{ or } \alpha) \rightarrow (x \text{ or } y \text{ or } \beta) \text{ and } (z \text{ or } \beta \text{ or } \gamma) \text{ and } (\alpha \text{ or } \beta \text{ or } \gamma)$$

Esto tiene como complejidad $O(|C|)$ siendo C el conjunto de cláusulas de la instancia de NAE-4-SAT. Los valores asignados a β , γ buscarán preservar la condición de "al menos uno TRUE y los demás FALSE o viceversa".

Ya hemos demostrado que NAE-3-SAT es NP-Hard, ahora resta demostrar que es NP. Para ello lo que podemos hacer es un análisis igual al enunciado para la validación de la instancia de NAE-4-SAT del punto anterior y por lo tanto este validador será $O(|C|)$ en tiempo de ejecución siendo C el conjunto de cláusulas de la instancia de NAE-3-SAT.

Así queda demostrado que NAE-3-SAT es NP completo.

4. Reducción de NAE-3-SAT a Set Splitting:

Este análisis queda igual a la primera entrega.

5. Transformación de la solución de Set Splitting a la solución de la instancia construida de NAE-3-SAT:

Este análisis queda igual a la primera entrega.

6. Transformación de la solución de NAE-3-SAT la solución de nuestra instancia construida de NAE-4-SAT:

Los valores de los literales en la solución de la instancia construida para NAE-3-SAT corresponden a una solución de la instancia construida de NAE-4-SAT. Recordemos que los literales β , γ no existen en la instancia de NAE-4-SAT pero esto no nos representará una dificultad dado que los literales originales de la instancia de NAE-4-SAT tendrán el mismo valor que los literales de la solución de la instancia construida para NAE-3-SAT. Por lo tanto la complejidad de esta transformación es $O(K)$ con K la cantidad de literales en NAE-3-SAT ya que podemos guardar en un diccionario sus valores para luego recorrer las cláusulas de NAE-4-SAT y reemplazar estos valores.

7. Transformación de la solución de NAE-4-SAT la solución de nuestra instancia original de 3-SAT:

Los valores de los literales en la solución de la instancia construida para NAE-4-SAT corresponden a una solución de la instancia original de 3-SAT de la cual partimos. Recordemos que "separamos" las cláusulas originales para poder aplicar la restricción de "no todos iguales" la cual ahora no tenemos, por lo tanto los mismos valores de los literales que solucionan la instancia construida de NAE-4-SAT también solucionan la instancia original de 3-SAT porque por definición de NAE-4-SAT sabemos que al menos un literal tiene valor TRUE.

Pregunta 5: Un tercer problema al que llamaremos X se puede reducir polinomialmente al problema de “Las 2 jornadas de capacitación”, qué podemos decir acerca de su complejidad?

Si un problema X se reduce en tiempo polinomial al problema de las 2 jornadas de capacitación ($P2J$), es decir $X <_p P2J$, entonces podemos afirmar que $P2J$ **es al menos tan difícil como** X . Es decir, si tuviéramos una caja negra que resuelva cualquier instancia de $P2J$, también podríamos resolver una instancia de X mediante la misma tras una transformación.

Sin embargo, esta reducción **no nos permite clasificar a** X . Aunque sepamos que $P2J$ es NP-Completo, **no podemos afirmar que** X **sea NP, NP-Hard ni mucho menos NP-Completo**, ya que no tenemos información adicional sobre su complejidad.